

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Campus Betim

Curso: Sistemas de Informação

Disciplina: Sistemas Operacionais

Trabalho Prático 2

Projetando um Gerenciador de Memória Virtual

Integrantes

Ana Luiza Damasceno Miranda

João Vitor Alves Amaral

João Vitor Vieira Guedes

Marcus Magnus de Almeida Mayer

Pedro Martins Assunção de Oliveira

Professor: Lucas Bragança da Silva

Betim

06/2026

1. Introdução

A gerência de memória é um dos componentes centrais de qualquer sistema operacional moderno, sendo responsável por permitir que múltiplos processos compartilhem um recurso físico limitado de forma segura, eficiente e transparente. Para viabilizar esse compartilhamento, os sistemas operacionais empregam a técnica de memória virtual, que desacopla o espaço de endereçamento utilizado pelos programas (endereços lógicos) do espaço real de armazenamento na memória RAM (endereços físicos).

Este trabalho tem como objetivo o desenvolvimento de um simulador, em linguagem C, capaz de reproduzir o processo de tradução de endereços lógicos para endereços físicos em um sistema com paginação. O simulador implementa os principais mecanismos envolvidos nesse processo: uma Translation Lookaside Buffer (TLB), responsável por acelerar traduções recentes; uma tabela de páginas, que mantém o mapeamento completo entre páginas virtuais e quadros físicos; o tratamento de faltas de página (page faults), que ocorre quando uma página requisitada ainda não está carregada na memória física; e uma política de substituição de páginas baseada no algoritmo de envelhecimento (Aging), uma aproximação eficiente do clássico algoritmo Least Recently Used (LRU).

O desenvolvimento deste projeto tem como propósito consolidar, na prática, os conceitos teóricos apresentados na disciplina de Sistemas Operacionais, em especial os mecanismos de tradução de endereços, hierarquia de cache de tradução e políticas de substituição de páginas. Por meio da implementação e dos testes realizados com diferentes padrões de acesso à memória, busca-se observar empiricamente o comportamento desses mecanismos e compará-los ao comportamento teórico esperado.

Este relatório está organizado de forma a apresentar, nas seções seguintes, a fundamentação teórica que embasa o projeto, a arquitetura e as estruturas de dados utilizadas, as decisões de implementação adotadas pelo grupo, os testes realizados e os resultados obtidos, além de uma seção dedicada à descrição do uso de ferramentas de Inteligência Artificial ao longo do desenvolvimento, em conformidade com as diretrizes estabelecidas no enunciado do trabalho.

2. Fundamentação Teórica

2.1 Memória Virtual e Paginação

A memória virtual é uma técnica que permite que um processo opere como se tivesse acesso a um espaço de endereçamento maior e isolado, independentemente da quantidade real de memória física disponível no sistema. Isso é possível porque o espaço de endereçamento virtual de cada processo é dividido em blocos de tamanho fixo, denominados páginas, enquanto a memória física é dividida em blocos de mesmo tamanho, denominados quadros (frames). O sistema operacional mantém, para cada processo, uma tabela de páginas que associa cada página virtual a um quadro físico correspondente, quando esta está carregada na memória.

Neste projeto, o espaço de endereçamento virtual possui 65.536 bytes (2^{16}), dividido em 256 páginas de 256 bytes cada. A memória física, por sua vez, possui 32.768 bytes, organizados em 128 quadros de 256 bytes. Como o espaço virtual é maior que o espaço físico disponível, nem todas as páginas podem estar residentes na memória ao mesmo tempo, o que torna necessário um mecanismo de substituição de páginas.

2.2 Tradução de Endereços

Cada endereço lógico gerado por um processo é dividido em duas partes: um número de página, que identifica qual página virtual está sendo referenciada, e um deslocamento (offset), que identifica a posição exata dentro daquela página. Como as páginas e os quadros possuem o mesmo tamanho, o deslocamento dentro da página é idêntico ao deslocamento dentro do quadro físico correspondente. A tradução consiste, portanto, em descobrir em qual quadro físico a página requisitada está armazenada, e então combinar esse número de quadro com o deslocamento original para obter o endereço físico final.

2.3 Translation Lookaside Buffer (TLB)

Consultar a tabela de páginas a cada acesso à memória implica em uma penalidade de desempenho, já que essa tabela normalmente reside na própria memória principal. Para mitigar esse custo, os processadores modernos utilizam uma TLB, uma memória associativa de alta velocidade que armazena as traduções mais recentemente utilizadas. Quando uma página solicitada está presente na TLB (TLB hit), a tradução é obtida quase instantaneamente. Quando não está (TLB miss), torna-se necessário consultar a tabela de páginas. Neste projeto, a TLB possui 16 entradas e adota a política de substituição FIFO (First In, First Out), na qual a entrada mais antiga é substituída quando a TLB está cheia.

2.4 Falta de Página (Page Fault) e Paginação por Demanda

Quando uma página referenciada não está presente nem na TLB nem na tabela de páginas, ocorre uma falta de página (page fault). Nesse caso, o sistema precisa localizar a página solicitada em um meio de armazenamento secundário — neste projeto representado pelo arquivo `BACKING_STORE.bin` — e carregá-la em um quadro disponível na memória física. Esse mecanismo é conhecido como paginação por demanda, pois as páginas são trazidas para a memória apenas quando efetivamente necessárias.

2.5 Substituição de Páginas e o Algoritmo de Envelhecimento (Aging)

Quando ocorre uma falta de página e não há quadros livres na memória física, é necessário escolher uma página já residente para ser removida. O algoritmo Least Recently Used (LRU) seleciona a página que não é utilizada há mais tempo, mas sua implementação exata é custosa. O algoritmo de Aging é uma aproximação eficiente do LRU: cada página mantém um bit de referência e um contador de envelhecimento de 8 bits. Sempre que a página é acessada, seu bit de referência é definido como 1. Periodicamente, todos os contadores são deslocados uma posição à direita e o bit de referência é inserido no bit mais significativo do contador, sendo

em seguida zerado. Contadores com valores menores indicam páginas que não são referenciadas há mais tempo, sendo essas as escolhidas para substituição.

3. Arquitetura e Estruturas de Dados

3.1 Organização do Projeto

O simulador foi organizado em cinco módulos independentes, cada um responsável por uma parte específica do processo de tradução de endereços. Essa separação facilita o desenvolvimento paralelo entre os integrantes do grupo e torna o código mais legível e testável de forma isolada. Os módulos são: `main.c` (ponto de entrada e fluxo principal), `tlb.c` (Translation Lookaside Buffer), `page_table.c` (tabela de páginas), `memory.c` (memória física e tratamento de page faults) e `statistics.c` (contagem e exibição de estatísticas). Os cabeçalhos correspondentes ficam no diretório `include/`, e os parâmetros globais do simulador estão centralizados em `config.h`.

3.2 Estruturas de Dados

Duas estruturas principais organizam os dados de mapeamento do simulador:

A estrutura `tlb_entry_t` representa uma entrada da TLB, composta pelo número da página (`page`), o número do quadro físico correspondente (`frame`) e um bit de validade (`valid`) que indica se a entrada contém dados válidos.

A estrutura `page_table_entry_t` representa uma entrada da tabela de páginas. Além dos campos `frame` e `valid`, ela contém dois campos adicionais para suporte ao algoritmo de Aging: `reference_bit`, definido como 1 sempre que a página é acessada, e `aging_counter`, um contador de 8 bits atualizado periodicamente para aproximar o comportamento do LRU.

3.3 Fluxo de Tradução de Endereços

Para cada endereço lógico lido da entrada, o simulador executa o seguinte fluxo:

- O endereço é mascarado com `0xFFFF` para considerar apenas os 16 bits menos significativos. Os 8 bits superiores formam o número da página e os 8 bits inferiores formam o offset.
- A TLB é consultada. Se a página for encontrada (TLB hit), o quadro é obtido diretamente e o contador de TLB hits é incrementado.
- Em caso de TLB miss, a tabela de páginas é consultada. Se a página for válida, o quadro é obtido da tabela.
- Se a página não estiver na tabela de páginas (page fault), ela é carregada do arquivo `BACKING_STORE.bin` para um quadro da memória física. Se não houver quadro livre, o algoritmo de Aging seleciona a página vítima.
- A entrada na TLB é atualizada com o novo mapeamento. O bit de referência da página é marcado como 1 e o aging é atualizado.

- O endereço físico é calculado como $\text{frame} * 256 + \text{offset}$, o byte correspondente é lido da memória física e impresso junto com os endereços lógico e físico.

4. Descrição da Implementação e Decisões de Projeto

4.1 Extração de Página e Offset (main.c)

O endereço lógico recebido é um inteiro de 32 bits, mas apenas os 16 bits menos significativos são relevantes. A extração é feita com operações de máscara de bits e deslocamento:

```
logical_address = logical_address & 0xFFFF;
int page      = (logical_address >> 8) & 0xFF;
int offset    = logical_address & 0xFF;
```

O endereço físico é calculado combinando o número do quadro obtido na tradução com o offset:

```
int physical_address = frame * FRAME_SIZE + offset;
```

4.2 TLB com Política FIFO (tlb.c)

A TLB foi implementada como um vetor estático de 16 entradas. A função `tlb_insert` segue três prioridades: (1) se a página já existe na TLB, apenas atualiza o quadro; (2) se há uma entrada inválida disponível, ocupa essa posição; (3) se a TLB está cheia, substitui a entrada mais antiga usando um índice FIFO circular (`fifo_next`), que avança de 0 a 15 com operação módulo. A função `tlb_remove` invalida a entrada correspondente à página informada, sendo chamada sempre que uma página é removida da memória física.

4.3 Tabela de Páginas e Algoritmo de Aging (page_table.c)

A tabela de páginas é um vetor estático de 256 entradas do tipo `page_table_entry_t`. A função `page_table_lookup` retorna o quadro se a entrada for válida, ou -1 caso contrário. A função `page_table_update` registra o mapeamento e zera os campos de controle do LRU. A função `page_table_invalidate` é chamada quando uma página vítima é removida da memória.

O algoritmo de Aging é implementado em `page_table_update_aging`, que percorre todas as páginas válidas a cada acesso à memória: o `aging_counter` é deslocado um bit à direita, o `reference_bit` é inserido no bit mais significativo (usando a máscara `0x80`) e em seguida zerado. Isso garante que páginas acessadas frequentemente mantenham contadores com valores maiores, enquanto páginas não utilizadas têm seus contadores progressivamente reduzidos.

4.4 Tratamento de Page Faults e Substituição (memory.c)

A função `handle_page_fault` busca primeiro um quadro livre no vetor `frame_to_page`. Se não houver quadro livre, `select_victim_page` percorre todas as páginas válidas e seleciona a que possui o menor `aging_counter`. O quadro da página vítima é obtido, ela é invalidada na tabela de páginas e removida da TLB.

Em seguida, a página solicitada é lida do BACKING_STORE.bin com fseek e fread, posicionando o cursor em page * PAGE_SIZE para leitura exata de 256 bytes. O vetor frame_to_page e a tabela de páginas são então atualizados com o novo mapeamento.

4.5 Estatísticas (statistics.c)

O módulo de estatísticas mantém três contadores inteiros estáticos: total_addresses, page_faults e tlb_hits. Cada contador é incrementado pela função correspondente chamada no fluxo principal. As taxas são calculadas como divisão de ponto flutuante no momento da impressão, com proteção contra divisão por zero verificando se total_addresses é maior que zero antes de efetuar o cálculo.

5. Divisão de Trabalho

O desenvolvimento do projeto foi dividido entre os integrantes do grupo de acordo com os arquivos-fonte do simulador, de forma a equilibrar a complexidade de implementação e permitir o desenvolvimento em paralelo. A divisão adotada está descrita na Tabela 1.

Integrante	Arquivo	Descrição
Ana Luiza Damasceno Miranda	Documentação	Elaboração do relatório técnico, estruturação das seções e documentação do uso de IA
João Vitor Alves Amaral	Documentação	Elaboração do relatório técnico, revisão do conteúdo e documentação dos prompts utilizados
João Vitor Vieira Guedes	page_table.c	Implementação da tabela de páginas e do mecanismo de Aging
Marcus Magnus de Almeida Mayer	memory.c e statistics.c	Tratamento de page faults, substituição de páginas, leitura da memória física e implementação dos contadores de estatísticas
Pedro Martins Assunção de Oliveira	tlb.c e main.c	Implementação da TLB com política FIFO, extração de página/offset e integração dos módulos

Tabela 1: Divisão de responsabilidades entre os integrantes do grupo.

Cada integrante ficou responsável pela implementação completa de um arquivo-fonte. Independentemente da divisão, todos os integrantes participaram da revisão do código, da integração entre os módulos e da execução dos testes finais.

6. Testes Realizados e Análise dos Resultados

6.1 Metodologia de Testes

O código-fonte completo do projeto está disponível no repositório público do GitHub: <https://github.com/Jones-Amaral/Trabalho-Pratico-2>.

Os testes foram realizados em duas etapas. Na primeira, cada módulo foi validado de forma isolada com casos de teste específicos, cobrindo cenários de sucesso, falha e condições de borda (como índices fora do intervalo e TLB cheia). Na segunda etapa, o projeto foi compilado e executado de forma integrada com os dois arquivos de endereços fornecidos: `addresses_random.txt`, com 10.000 endereços gerados aleatoriamente, e `addresses_location.txt`, com 10.000 endereços gerados com localidade de referência, nos quais 85% dos acessos se concentram em páginas próximas à página atual.

6.2 Resultados Obtidos

Os resultados das execuções integradas estão apresentados na Tabela 2.

Métrica	Endereços Aleatórios	Localidade de Referência
Total de endereços traduzidos	10.000	10.000
Page Faults	4.958	1.164
Page Fault Rate	49,6%	11,6%
TLB Hits	654	7.925
TLB Hit Rate	6,5%	79,2%

Tabela 2: Resultados das execuções com os dois arquivos de endereços.

Os resultados apresentados correspondem às execuções finais realizadas após a implementação e integração completa dos módulos do simulador, considerando os cenários de acesso aleatório e com localidade de referência.

As Figuras 1 e 2 apresentam os resultados reais da execução do simulador em ambiente Linux, confirmando os valores registrados na tabela acima.


```
20 de jun 22:26
pedroques:/TrabalhoPr-tico02_SO-main

Logical address: 11274 Physical address: 17418 Value: 10
Logical address: 3177 Physical address: 25193 Value: 105
Logical address: 12699 Physical address: 12865 Value: 65
Logical address: 26078 Physical address: 3382 Value: 54
Logical address: 4356 Physical address: 26372 Value: 4
Logical address: 18423 Physical address: 17399 Value: -9
Logical address: 686 Physical address: 11950 Value: -81
Logical address: 47128 Physical address: 27160 Value: 24
Logical address: 53488 Physical address: 31464 Value: -24
Logical address: 9432 Physical address: 17624 Value: -40
Logical address: 44657 Physical address: 5745 Value: 113
Logical address: 24288 Physical address: 25312 Value: -32
Logical address: 65834 Physical address: 25354 Value: 10
Logical address: 24921 Physical address: 26457 Value: 89
Logical address: 27194 Physical address: 12858 Value: 58
Logical address: 62093 Physical address: 38861 Value: -115
Logical address: 46151 Physical address: 22887 Value: 71
Logical address: 46725 Physical address: 3973 Value: -123
Logical address: 18071 Physical address: 11927 Value: -105
Logical address: 8892 Physical address: 17566 Value: 98
Logical address: 57524 Physical address: 27828 Value: -76
Logical address: 56834 Physical address: 7138 Value: -30
Logical address: 49298 Physical address: 28386 Value: -110
Logical address: 9288 Physical address: 17224 Value: 72
Logical address: 18242 Physical address: 25090 Value: 2
Logical address: 16671 Physical address: 26399 Value: 31
Logical address: 39889 Physical address: 22961 Value: -79
Logical address: 13226 Physical address: 11946 Value: -86
Logical address: 15450 Physical address: 17504 Value: 96
Logical address: 65367 Physical address: 16215 Value: 87
Logical address: 30221 Physical address: 20749 Value: 13
Logical address: 6747 Physical address: 3419 Value: 91
Logical address: 43374 Physical address: 29886 Value: 110
Logical address: 2954 Physical address: 17290 Value: -118
Logical address: 29302 Physical address: 25266 Value: -78
Logical address: 37997 Physical address: 18861 Value: 109
Logical address: 32949 Physical address: 11957 Value: -75
Logical address: 51173 Physical address: 19173 Value: -27
Logical address: 46911 Physical address: 7231 Value: 63
Logical address: 8986 Physical address: 17616 Value: -54
Logical address: 26891 Physical address: 8538 Value: 125
Logical address: 37757 Physical address: 13181 Value: 125
Logical address: 9801 Physical address: 17225 Value: 73
Logical address: 21640 Physical address: 26520 Value: -104
Logical address: 48925 Physical address: 27613 Value: -35
Logical address: 52326 Physical address: 18822 Value: 102
Logical address: 7424 Physical address: 12880 Value: 0
Logical address: 14532 Physical address: 25284 Value: -69
Logical address: 61732 Physical address: 29988 Value: 36
Logical address: 34478 Physical address: 17582 Value: -82
Logical address: 3929 Physical address: 3417 Value: 89
Logical address: 4238 Physical address: 17286 Value: -122
Logical address: 44388 Physical address: 28692 Value: 20
Logical address: 55234 Physical address: 38658 Value: -62
Logical address: 34194 Physical address: 7826 Value: -110
Number of Translated Addresses = 10000
Page Faults = 4958
Page Fault Rate = 0.496
TLB Hits = 654
TLB Hit Rate = 0.065

pedroques matrixlenovo ../TrabalhoPr-tico02_SO-main
```

Figura 1: Execução do simulador com addresses_random.txt em ambiente Linux.

```
20 de jun 22:27
pedroques:/TrabalhoPr-tico02_SO-main

Logical address: 48403 Physical address: 25555 Value: -45
Logical address: 48304 Physical address: 25456 Value: 112
Logical address: 41387 Physical address: 6827 Value: -85
Logical address: 48543 Physical address: 23301 Value: 95
Logical address: 48894 Physical address: 38910 Value: -66
Logical address: 41292 Physical address: 6732 Value: 76
Logical address: 41166 Physical address: 8422 Value: -59
Logical address: 41281 Physical address: 6721 Value: 65
Logical address: 48667 Physical address: 23515 Value: -37
Logical address: 41119 Physical address: 8375 Value: -97
Logical address: 41288 Physical address: 9464 Value: -8
Logical address: 48950 Physical address: 38966 Value: -10
Logical address: 41425 Physical address: 6865 Value: -47
Logical address: 23899 Physical address: 22571 Value: 43
Logical address: 41074 Physical address: 8330 Value: 114
Logical address: 48786 Physical address: 38802 Value: 82
Logical address: 48325 Physical address: 25477 Value: -123
Logical address: 26823 Physical address: 26823 Value: -57
Logical address: 48540 Physical address: 23388 Value: 92
Logical address: 45352 Physical address: 15480 Value: 48
Logical address: 48772 Physical address: 38788 Value: 68
Logical address: 48253 Physical address: 25405 Value: 61
Logical address: 41184 Physical address: 9360 Value: -112
Logical address: 41221 Physical address: 6661 Value: 9
Logical address: 41159 Physical address: 9415 Value: -57
Logical address: 48238 Physical address: 25390 Value: 46
Logical address: 41433 Physical address: 8873 Value: -39
Logical address: 48968 Physical address: 8224 Value: 8
Logical address: 48881 Physical address: 38897 Value: -79
Logical address: 48982 Physical address: 9218 Value: 2
Logical address: 48536 Physical address: 23384 Value: 88
Logical address: 48403 Physical address: 25555 Value: -45
Logical address: 48917 Physical address: 38933 Value: -43
Logical address: 48626 Physical address: 22476 Value: -76
Logical address: 41151 Physical address: 9407 Value: -65
Logical address: 41364 Physical address: 6884 Value: -108
Logical address: 41374 Physical address: 6814 Value: -98
Logical address: 48344 Physical address: 25496 Value: -104
Logical address: 48383 Physical address: 25455 Value: 111
Logical address: 28976 Physical address: 22768 Value: -16
Logical address: 48383 Physical address: 25455 Value: 111
Logical address: 16797 Physical address: 15005 Value: -99
Logical address: 23011 Physical address: 26851 Value: -29
Logical address: 41865 Physical address: 8321 Value: 106
Logical address: 42561 Physical address: 19265 Value: 65
Logical address: 38126 Physical address: 3246 Value: -82
Logical address: 48251 Physical address: 25483 Value: 59
Logical address: 41293 Physical address: 6733 Value: 77
Logical address: 48449 Physical address: 23297 Value: 1
Logical address: 56232 Physical address: 32168 Value: -88
Logical address: 41831 Physical address: 9287 Value: 71
Logical address: 48363 Physical address: 25515 Value: -85
Logical address: 48244 Physical address: 25390 Value: 52
Logical address: 48408 Physical address: 23336 Value: 48
Logical address: 48975 Physical address: 9231 Value: 15
Number of Translated Addresses = 10000
Page Faults = 1164
Page Fault Rate = 0.116
TLB Hits = 7925
TLB Hit Rate = 0.792

pedroques matrixlenovo ../TrabalhoPr-tico02_SO-main
```

Figura 2: Execução do simulador com addresses_location.txt em ambiente Linux.

6.3 Análise dos Resultados

Os resultados obtidos estão em plena conformidade com o comportamento teórico esperado para cada padrão de acesso à memória.

No cenário de endereços aleatórios, a taxa de page fault de 49,6% reflete o fato de que os acessos são distribuídos uniformemente entre as 256 páginas virtuais. Como a memória física comporta apenas 128 quadros, aproximadamente metade das páginas não pode estar residente ao mesmo tempo, resultando em um alto número de substituições. A TLB hit rate de apenas 6,5% é coerente com esse cenário: como as páginas acessadas variam aleatoriamente, a probabilidade de que uma página já consultada reapareça nas próximas 16 requisições (tamanho da TLB) é baixa.

No cenário com localidade de referência, os resultados melhoram drasticamente. A taxa de page fault cai para 11,6%, pois os acessos se concentram em um conjunto reduzido de páginas próximas, que tendem a permanecer residentes na memória física ao longo do tempo. A TLB hit rate sobe para 79,2%, confirmando que a localidade de referência permite que a TLB de 16 entradas cubra a grande maioria das traduções necessárias. Esses resultados validam empiricamente o princípio da localidade, que é um dos fundamentos teóricos do funcionamento eficiente de TLBs e cachês em sistemas reais.

7. Uso de Inteligência Artificial

7.1 Ferramentas Utilizadas

O grupo utilizou o assistente de Inteligência Artificial Claude (Anthropic) ao longo do desenvolvimento do projeto. O uso foi registrado conforme as diretrizes do enunciado, seguindo o modelo Spec-Driven Development (SDD): os prompts definem claramente o problema, especificam requisitos funcionais e não funcionais, indicam restrições de implementação e evitam solicitações vagas.

Além da utilização principal do Claude durante o desenvolvimento, outras ferramentas de IA foram utilizadas pontualmente como apoio à revisão textual, verificação de clareza das explicações e validação da documentação produzida. Essas ferramentas não foram utilizadas para substituir decisões de implementação nem para geração automática da solução final, tendo sido empregadas apenas como suporte complementar ao processo de desenvolvimento e revisão.

7.2 Metodologia de Uso

Os prompts utilizados durante o desenvolvimento foram feitos de forma natural durante as sessões de trabalho. Como o enunciado exige o formato Spec-Driven Development (SDD), o grupo utilizou o próprio Claude para auxiliar na reformulação dos prompts técnicos para esse formato, conforme autorizado pelo professor. Para os prompts de entendimento de conceitos e documentação, foi mantida apenas a versão natural, por refletirem perguntas genuinamente conversacionais. Para os prompts técnicos de implementação, testes e depuração, são apresentadas as duas versões.

7.3 Prompts Gerais de Entendimento

Categoria 1 — Entendimento do Trabalho

"Tenho um trabalho de SO para fazer um simulador de memória virtual em C. O professor deu uns arquivos base com várias funções marcadas como TODO para a gente implementar. Pode me explicar o que cada parte faz e por onde eu começo?"

Categoria 2 — Entendimento de Conceitos da Matéria

"Não entendo o que é offset, TLB, page fault e LRU. Pode me explicar esses termos de um jeito simples?"

Categoria 3 — Entender o Código Existente e os TODOs

"Recebi esse arquivo do meu professor e não entendi o que as funções fazem e o que preciso implementar nos TODOs. Pode me explicar?"

Categoria 4 — Entender Termos e Operações do Código

"Tem umas coisas no código que não entendo: aging_counter, a operação $\mid= 0x80$, o campo valid, o fifo_next e o fseek/fread. Pode explicar?"

Categoria 5 — Documentação e Relatório

"Preciso fazer um relatório para esse trabalho. Pode me ajudar a montar a estrutura com as seções que o professor pediu?"

Categoria 6 — Reformulação dos Prompts para o Formato SDD

"Meu professor pediu para documentar todos os prompts no formato SDD. Pode me ajudar a reescrever os que usei de forma mais formal?"

7.4 Prompts Técnicos de Implementação (com versão SDD)

Categoria 7 — Testes e Validação do Código

Versão natural:

"Pode testar esse código pra mim e verificar se está correto?"

Versão SDD:

"Implementei a função [nome da função] em C para um simulador de memória virtual. Preciso validar se a implementação está correta através de casos de teste que cubram: (1) comportamento esperado em cenário normal; (2) condições de borda como índices inválidos; (3) comportamento quando estruturas estão vazias ou cheias. Linguagem: C (C11)."

Categoria 8 — Depuração de Erros

Versão natural:

"Rodei o programa e o resultado saiu tudo zero, o que pode ser?"

Versão SDD:

"Estou depurando um simulador de memória virtual em C. Após compilar e executar, todos os endereços físicos e valores retornam zero, e as estatísticas mostram Page Fault Rate = 1.000 e TLB Hit Rate = 0.000. Preciso identificar qual módulo está com problema e qual a causa mais provável desse comportamento incorreto, considerando que os arquivos foram atualizados parcialmente."

7.5 Prompts por Integrante

A seguir estão os prompts técnicos utilizados por cada integrante durante a implementação do seu respectivo arquivo, nas versões natural e SDD.

Ana Luiza Damasceno e João Vitor Alves — Documentação

"Preciso fazer um relatório para esse trabalho de SO. O professor pediu introdução, fundamentação teórica, arquitetura, implementação, divisão de trabalho, testes e resultados, uso de IA e conclusão. Pode me ajudar a montar e redigir cada seção?"

"O relatório precisa ter uma capa com os dados do grupo. Como monto isso no Word de forma acadêmica?"

"Meu professor pediu que os prompts fossem separados ao invés de condensados. Pode me ajudar a reorganizar a seção de uso de IA do relatório?"

"Meu colega acha que vale a pena colocar prints mostrando que o programa rodou no Linux. Onde fica melhor colocar isso no relatório?"

"O professor pediu o link do repositório no relatório. Em que parte acha melhor colocar?"

Como foi utilizado: a IA auxiliou na estruturação e redação do relatório técnico, sugerindo a organização das seções conforme os critérios de avaliação do enunciado, redigindo o conteúdo das seções de introdução, fundamentação teórica, arquitetura, implementação, testes e conclusão. Também foi utilizada para reorganizar a seção de uso de IA conforme o feedback do professor, reformular os prompts naturais para o formato SDD e orientar sobre as melhores práticas de entrega, como o uso do `.gitignore` e a inclusão do link do repositório no relatório. Todo o conteúdo gerado foi revisado, adaptado e validado pelas integrantes responsáveis pela documentação antes de ser incorporado ao relatório final.

João Vitor Vieira Guedes — page_table.c

Versão natural:

"No page_table.c tem vários TODOs: lookup, update, invalidate, set_reference e o aging. Pode me explicar o que cada um faz e como implementar o aging com os bits?"

Versão SDD:

"Estou implementando um simulador de memória virtual em C. O arquivo page_table.c gerencia uma tabela de páginas com 256 entradas do tipo page_table_entry_t, com campos frame, valid, reference_bit e aging_counter. Preciso implementar: (1) page_table_lookup — retorna o frame se valid == 1, senão retorna -1; (2) page_table_update — define frame, valid = 1, zera reference_bit e aging_counter; (3) page_table_invalidate — define valid = 0 e frame = -1; (4) page_table_set_reference — define reference_bit = 1 se a página for válida; (5) page_table_update_aging — para cada página válida, desloca aging_counter um bit à direita, insere reference_bit no bit mais significativo (0x80) e zera reference_bit. Linguagem: C (C11)."

Como foi utilizado: a IA explicou o funcionamento do algoritmo de Aging com operações de bits, o significado da máscara 0x80 e o comportamento esperado do contador ao longo do tempo. O integrante revisou e validou a implementação com testes isolados.

Marcus Magnus de Almeida Mayer — memory.c e statistics.c

Versão natural - memory.c:

"O memory.c tem o handle_page_fault, select_victim_page e read_memory para implementar. Como eu leio a página do arquivo .bin e como escolher qual página substituir quando a memória enche?"

Versão SDD:

"Estou implementando um simulador de memória virtual em C. O arquivo memory.c gerencia a memória física com 128 quadros de 256 bytes (physical_memory[NUM_FRAMES][FRAME_SIZE]) e um array frame_to_page[NUM_FRAMES] que indica qual página está em cada quadro (valor -1 significa quadro livre). Preciso implementar: (1) read_memory(int frame, int offset) — retorna physical_memory[frame][offset]; (2) select_victim_page() — percorre todas as páginas válidas usando page_table_is_valid e page_table_get_aging_counter, e retorna o número da página com menor aging_counter; (3) handle_page_fault(int page) — busca quadro livre com find_free_frame(), se não houver chama select_victim_page(), obtém o quadro da vítima com page_table_get_frame, invalida na tabela de páginas, remove do TLB, faz fseek(backing, page * PAGE_SIZE, SEEK_SET) e fread de PAGE_SIZE bytes

para `physical_memory[frame]`, atualiza `frame_to_page` e chama `page_table_update`. Linguagem: C (C11)."

Como foi utilizado: a IA explicou o uso de `fseek` e `fread` para acesso aleatório ao arquivo binário, a importância de verificar o retorno do `fread` e o fluxo completo de substituição de páginas. O integrante revisou e validou com testes que incluíram o cenário de memória cheia.

Versão natural - statistics.c :

"No `statistics.c` preciso implementar os contadores de endereços, page faults e TLB hits, e calcular as taxas no `print`. Como faço a divisão sem perder as casas decimais e sem dar erro quando não tiver nenhum endereço?"

Versão SDD:

"Estou implementando um simulador de memória virtual em C para uma disciplina de Sistemas Operacionais. O arquivo `statistics.c` possui três contadores inteiros estáticos (`total_addresses`, `page_faults`, `tlb_hits`) já declarados. Preciso implementar as funções `count_address`, `count_page_fault` e `count_tlb_hit` — cada uma deve apenas incrementar seu respectivo contador. A função `print_statistics` já imprime os valores, mas preciso calcular `page_fault_rate` e `tlb_hit_rate` como `double`, dividindo cada contador por `total_addresses`, com proteção contra divisão por zero. Linguagem: C (C11). Não devo alterar as assinaturas das funções nem as variáveis existentes."

Como foi utilizado: a IA explicou a lógica dos contadores, o motivo do `cast` para `double` na divisão e a necessidade de proteção contra divisão por zero. O código gerado foi revisado, compreendido e validado pelo integrante antes de ser integrado ao projeto.

Pedro Martins Assunção de Oliveira — main.c e tlb.c

Versão natural — main.c :

"No `main.c` têm um `TODO` pra extrair a página e o offset do endereço lógico e calcular o endereço físico. Como eu faço isso? O endereço tem 32 bits mas só os 16 menores importam."

Versão SDD:

"Estou implementando um simulador de memória virtual em C. No arquivo `main.c`, o endereço lógico é lido como inteiro de 32 bits, mas só os 16 bits menos significativos importam. Desses 16 bits, os 8 mais altos são o número da página e os 8 mais baixos são o offset. Preciso: (1) mascarar o endereço com `0xFFFF`, (2) extrair `page` com `shift` de 8 bits e máscara `0xFF`, (3) extrair `offset` com máscara `0xFF`, e (4) calcular `physical_address = frame * FRAME_SIZE + offset`. As constantes `FRAME_SIZE` estão definidas em `config.h`. Não devo alterar o restante da lógica do `main`."

Como foi utilizado: a IA explicou o funcionamento das operações de máscara de bits e deslocamento, e o motivo pelo qual o offset é idêntico nos endereços lógico e físico. O integrante revisou e validou as operações antes da integração.

Versão natural — tlb.c :

"Tenho que implementar a TLB com FIFO no tlb.c. Tem três funções: lookup para buscar, insert para inserir com FIFO e remove para invalidar. Como funcionam essas três?"

Versão SDD:

"Estou implementando um simulador de memória virtual em C. O arquivo tlb.c gerencia uma TLB com 16 entradas, cada uma do tipo tlb_entry_t com campos page, frame e valid. Preciso implementar três funções: (1) tlb_lookup(int page) — percorre as entradas e retorna o frame se encontrar a página com valid == 1, ou -1 se não encontrar; (2) tlb_insert(int page, int frame) — insere uma entrada usando política FIFO com a variável fifo_next, atualizando circularmente de 0 a TLB_SIZE-1, mas antes verificando se a página já existe para apenas atualizar o frame e se há entrada inválida disponível; (3) tlb_remove(int page) — invalida a entrada correspondente à página informada. Linguagem: C (C11). Não devo alterar as assinaturas nem a estrutura existente."

Como foi utilizado: a IA explicou o funcionamento do índice FIFO circular com operação módulo e as três prioridades de inserção. O integrante revisou a lógica e validou o comportamento com testes antes da integração.

8. Conclusão

Este trabalho propôs-se a consolidar, por meio da implementação prática de um simulador de gerência de memória virtual, conceitos centrais da disciplina de Sistemas Operacionais, como paginação, tradução de endereços, uso de TLB e políticas de substituição de páginas. Ao longo do desenvolvimento, o grupo pôde relacionar de forma concreta os conceitos teóricos estudados com seu comportamento real em código, observando empiricamente fenômenos como a relação entre localidade de referência e taxa de acertos da TLB, ou o impacto da política de substituição sobre a taxa de faltas de página.

Os resultados obtidos confirmaram o comportamento previsto pela teoria: o padrão de acesso com localidade de referência produziu uma taxa de TLB hit de 79,2% e uma taxa de page fault de apenas 11,6%, em contraste com os 6,5% e 49,6% registrados no cenário de acesso aleatório. Essa diferença evidencia a importância da localidade de referência para o desempenho de sistemas de memória virtual e valida as estruturas implementadas.

Do ponto de vista técnico, a principal dificuldade encontrada foi compreender a interdependência entre os módulos e garantir que a invalidação de páginas na tabela de páginas e na TLB ocorresse de forma consistente durante as substituições. A divisão do projeto em módulos independentes, com testes isolados para cada arquivo, foi fundamental para identificar e corrigir comportamentos incorretos antes da integração final.

Por fim, o projeto reforçou a relevância dos conceitos de gerência de memória para o desempenho de sistemas operacionais modernos, demonstrando na prática como mecanismos como a TLB e o algoritmo de Aging contribuem para tornar o acesso à memória virtual eficiente mesmo em cenários com recursos físicos limitados.

9. Referências

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Sistemas Operacionais: Conceitos e Aplicações**. Editora LTC.

ANTHROPIC. **Claude**. Assistente de IA utilizado como suporte ao desenvolvimento. Disponível em: <https://claude.ai>. Acesso em: 29 jun. 2026.

OPENAI. **ChatGPT**. Assistente de Inteligência Artificial utilizado como suporte complementar para revisão textual e verificação da documentação do relatório. Disponível em: <https://chat.openai.com>. Acesso em: 29 jun. 2026.

GRUPO TP2. **Trabalho Prático 2 — Gerenciador de Memória Virtual**. Repositório público do projeto. Disponível em: <https://github.com/Jones-Amaral/Trabalho-Pratico-2>.